

LECTURE 06

State Management

Keeping your app's data in sync with the UI

Instructor · Muhammad Sabah

Today's plan

01

What is state?

Data that changes over time

02

setState

The simple way — local widget state

03

Lifting state up

Sharing state between widgets

04

Provider

App-wide state, made simple

What is state?

Any data that can change while your app is running.

EXAMPLES OF STATE

- **Counter value** Goes up when user taps +
- **Text in a field** Changes as user types
- **Toggle switch** On / off
- **Logged-in user** Null when signed out, set when signed in
- **Items in cart** Grows / shrinks as user shops

REMEMBER

State management
is just about deciding

WHERE
the data lives.

setState — local state

Use setState when the data only matters inside one widget.

EXAMPLE

```
class _CounterState extends State<Counter> {  
  int count = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text('$count'),  
        ElevatedButton(  
          onPressed: () {  
            setState(() => count++);  
          },  
          child: Text('Add'),  
        ),  
      ],  
    );  
  }  
}
```

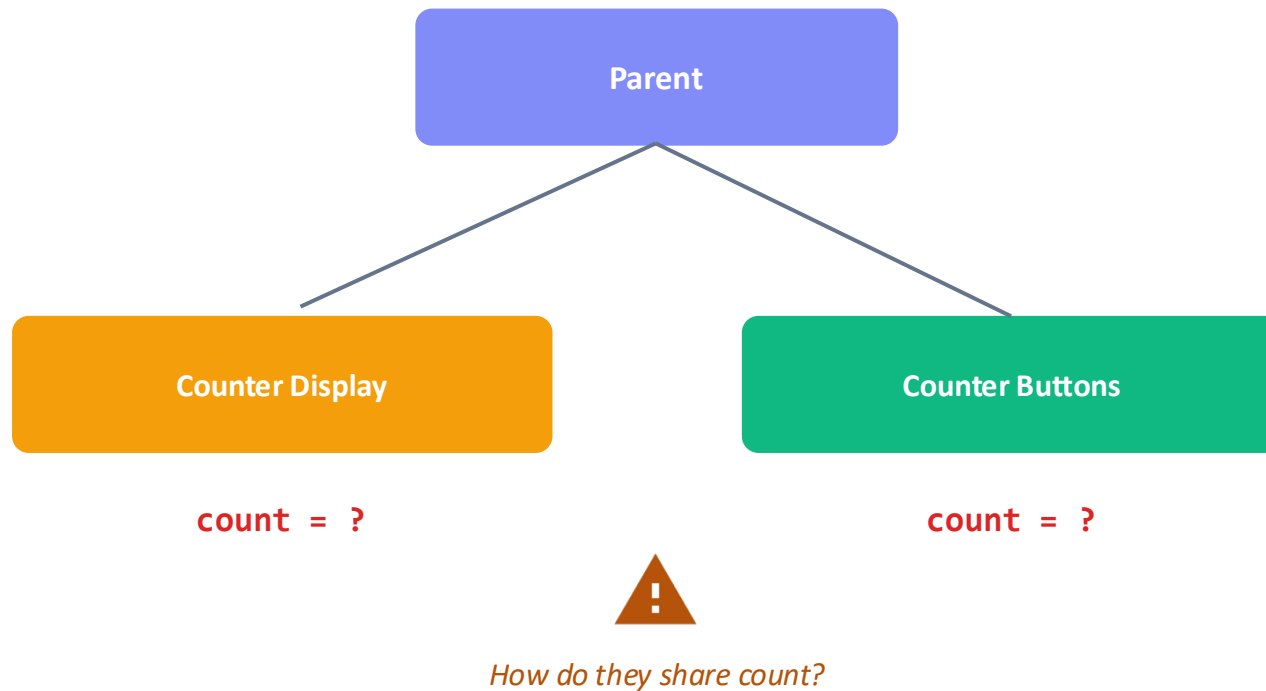
WHEN THIS WORKS

- ✓ Only one widget needs the data
- ✓ Data doesn't survive screen changes
- ✓ Toggle, counter, simple form

When setState isn't enough

Imagine two sibling widgets that need to share the same data.

TWO SIBLINGS, SAME DATA?



THE ISSUE

setState lives inside one widget.

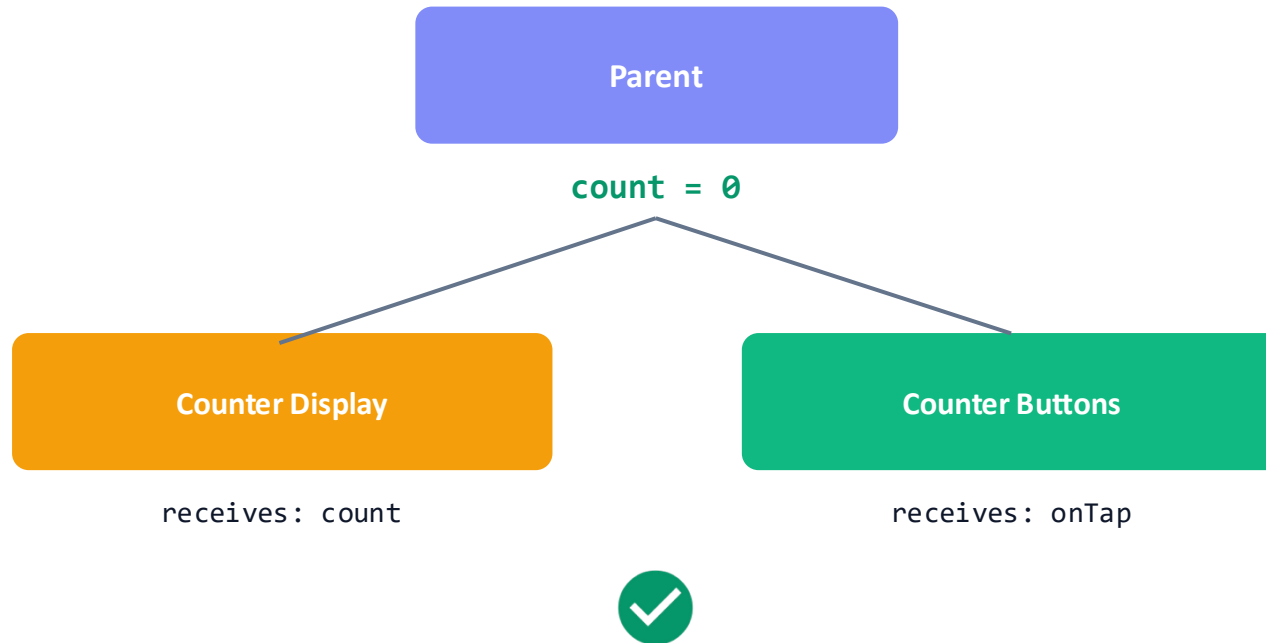
So if two widgets need the same data, neither one can own it alone — they need to agree on a single source.

→ **Lift the state up.**

Lifting state up

Move the state to the closest common parent. Pass it down to children.

STATE LIVES IN PARENT



IN CODE

```
// Parent holds the state
int count = 0;

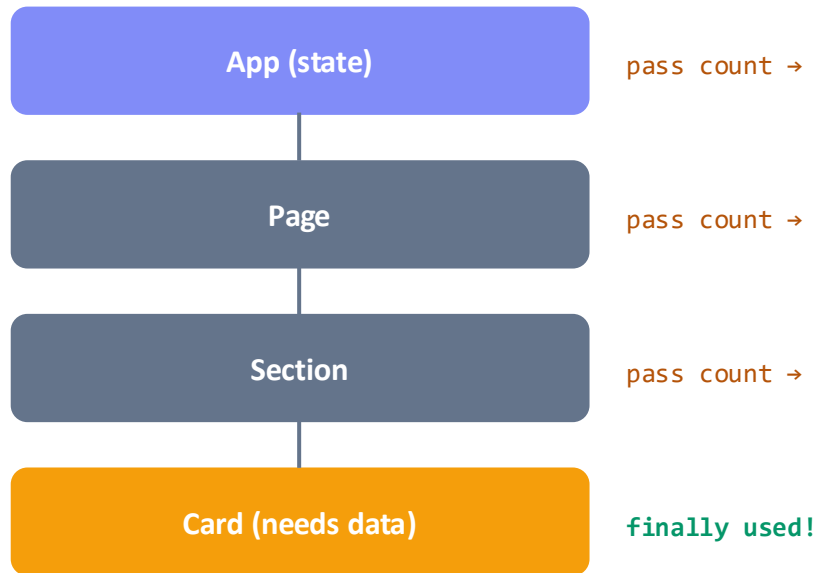
// Pass data down
CounterDisplay(count: count),

// Pass callback down
CounterButtons(
  onTap: () => setState(
    () => count++,
  ),
),
```

Lifting up gets messy

When data must reach deep widgets, you have to pass it through every level in between.

PROP DRILLING



WHY IT'S A PROBLEM

Tedious

Every middle widget needs the prop

Fragile

Forget one level, app breaks

Hard to share

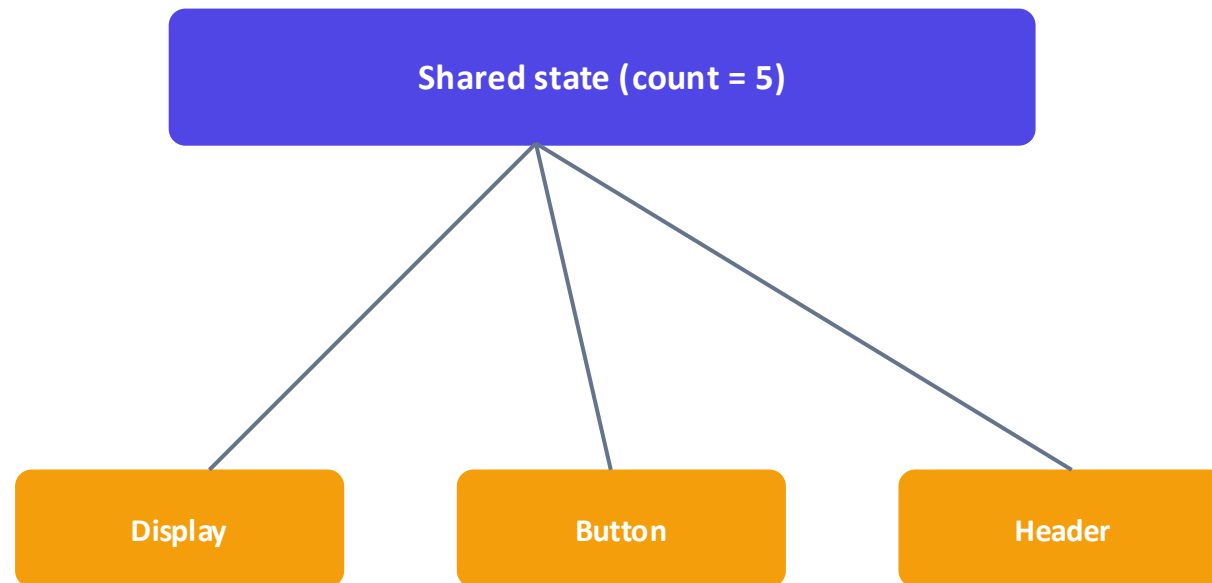
Distant widgets can't easily get it

→ Use Provider.

Meet Provider

A small package that lets any widget read shared state — without passing it through every level.

ANY WIDGET CAN READ IT



STEP 0 — INSTALL

```
# Add to pubspec.yaml
dependencies:
  provider: ^6.1.0

# Then run:
flutter pub get
```

THEN IMPORT

```
import 'package:provider/
  provider.dart';
```


Provider in 3 steps

1 Create a model class with ChangeNotifier

```
class CounterModel extends ChangeNotifier {  
  int count = 0;  
  
  void increment() {  
    count++;  
    notifyListeners(); // tell the UI to rebuild  
  }  
}
```

2 Wrap your app with ChangeNotifierProvider

```
ChangeNotifierProvider(  
  create: (context) => CounterModel(),  
  child: MyApp(),  
)
```

Step 3 — read & update from any widget

Inside any widget, ask for the model — read its value or call its methods.

READ — DISPLAY THE COUNT

```
@override
Widget build(BuildContext context) {
  final counter = context.watch<CounterModel>();

  return Text('Count: ${counter.count}');
}
```

UPDATE — TAP A BUTTON

```
ElevatedButton(
  onPressed: () {
    context.read<CounterModel>().increment();
  },
  child: Text('Add 1'),
)
```

The rule

context.watch — UI rebuilds when data changes (use in build).

context.read — call a method, no rebuild (use in callbacks like onPressed).

Build a shared counter with Provider

Two screens reading and updating the same counter value.

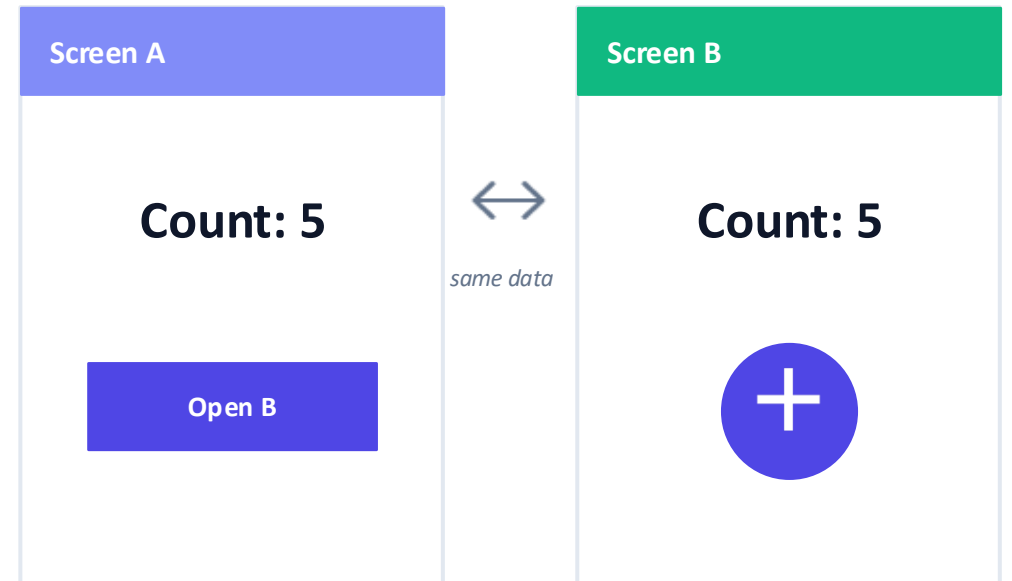
YOUR TASK

1. Add provider package to pubspec.yaml
2. Create CounterModel with ChangeNotifier
3. Wrap MyApp with ChangeNotifierProvider
4. Screen A: shows count + button to Screen B
5. Screen B: shows same count + add button

HINT

```
context.watch<CounterModel>().count // read  
context.read<CounterModel>().increment() // update
```

EXPECTED RESULT



W R A P U P

What we covered today

Y O U N O W K N O W

- ✓ What state is — data that changes
- ✓ setState for local widget state
- ✓ Lifting state up to a parent
- ✓ Why prop drilling gets painful
- ✓ Provider for shared, global state
- ✓ context.watch vs context.read

T H A N K Y O U

Questions?

Muhammad Sabah · hama-sabah.dev